

Problem 1: Finding Complexity using Counter Method

Convert the following algorithm into a program and find its time complexity using the counter method.

```
void function (int n)
{
    int i= 1;
    int s =1;
    while(s <= n)
    {
        i++;
        s += i;
    }
}
```

Note: No need of counter increment for declarations and scanf() and count variable printf() statements.

Input:

A positive Integer n

Output:

Print the value of the counter variable

For example:

Input	Result
9	12

Answer:(penalty regime: 0 %)

```
1  #include <stdio.h>
2
3  void function(int n)
4  {
5      int i = 1;
6      int s = 1;
7      int count = 0;
8
9      count++;
10     count++;
11
12     while (s <= n)
13     {
14         count++;
15
16         i++;
17         count++;
18
19         s += i;
20         count++;
21     }
22
23     count++;
24
25     printf("%d", count);
26 }
27
28 int main()
29 {
30     int n;
31     scanf("%d", &n);
32
33     function(n);
34
35     return 0;
36 }
```

	Input	Expected	Got	
✓	9	12	12	✓
✓	4	9	9	✓

Passed all tests! ✓

Correct

Marks for this submission: 1.00/1.00.

Problem 2: Finding Complexity using Counter method

Convert the following algorithm into a program and find its time complexity using the counter method.

```
void func(int n)
{
    if(n==1)
    {
        printf("");
    }
    else
    {
        for(int i=1; i<=n; i++)
        {
            for(int j=1; j<=n; j++)
            {
                printf("");
                printf("");
                break;
            }
        }
    }
}
```

Note: No need of counter increment for declarations and scanf() and count variable printf() statements.

Input:

A positive Integer n

Output:

Print the value of the counter variable

Answer:(penalty regime: 0 %)

```
1  #include <stdio.h>
2
3  int main() {
4      int n, counter = 0;
5
6      scanf("%d", &n);
7
8      counter++;
9
10     if (n == 1) {
11         counter++;
12     } else {
13         int i = 1;
14
15         while (i <= n) {
16             counter++;
17             counter++;
18             counter++;
19             counter++;
20
21             i++;
22             counter++;
23         }
24
25         counter++;
26     }
27
28     printf("%d", counter);
29
30     return 0;
31 }
```

	Input	Expected	Got	
✓	2	12	12	✓
✓	1000	5002	5002	✓
✓	143	717	717	✓

Passed all tests! ✓

Correct

Marks for this submission: 1.00/1.00.

Problem 3: Finding Complexity using Counter Method

Convert the following algorithm into a program and find its time complexity using counter method.

```
Factor(num) {  
  {  
    for (i = 1; i <= num;++i)  
    {  
      if (num % i== 0)  
      {  
        printf("%d ", i);  
      }  
    }  
  }  
}
```

Note: No need of counter increment for declarations and scanf() and counter variable printf() statement.

Input:

A positive Integer n

Output:

Print the value of the counter variable

```

1  #include <stdio.h>
2
3  int main() {
4      int num, i, counter = 0;
5
6      scanf("%d", &num);
7
8      for (i = 1; i <= num; ++i) {
9          counter++;
10
11         if (num % i == 0) {
12             counter++;
13         }
14
15         counter++;
16     }
17
18     counter++;
19
20     printf("%d", counter);
21
22     return 0;
23 }

```

	Input	Expected	Got	
✓	12	31	31	✓
✓	25	54	54	✓
✓	4	12	12	✓

Passed all tests! ✓

Correct

Marks for this submission: 1.00/1.00.

Problem 4: Finding Complexity using Counter Method

Convert the following algorithm into a program and find its time complexity using counter method.

```
void function(int n)
{
    int c= 0;
    for(int i=n/2; i<n; i++)
        for(int j=1; j<n; j = 2 * j)
            for(int k=1; k<n; k = k * 2)
                c++;
}
```

Note: No need of counter increment for declarations and scanf() and count variable printf() statements.

Input:

A positive Integer n

Output:

Print the value of the counter variable

```
1  #include <stdio.h>
2
3  int main() {
4      int n, c = 0;
5      int i, j, k;
6
7      scanf("%d", &n);
8
9      i = n / 2;
10     c++;
11
12     while (i < n) {
13         c++;
14
15         j = 1;
16
17         while (j < n) {
18             c++;
19
20             k = 1;
21
22             while (k < n) {
23                 c++;
24                 c++;
25
26                 k = k * 2;
27             }
```

```

29         c++;
30
31         j = 2 * j;
32     }
33
34     c++;
35
36     i++;
37 }
38
39 c++;
40
41 printf("%d", c);
42
43 return 0;
44 }

```

	Input	Expected	Got	
✓	4	30	30	✓
✓	10	212	212	✓

Passed all tests! ✓

Correct

Marks for this submission: 1.00/1.00.

Problem 5: Finding Complexity using counter method

Convert the following algorithm into a program and find its time complexity using counter method.

```
void reverse(int n)
{
    int rev = 0, remainder;
    while (n != 0)
    {
        remainder = n % 10;
        rev = rev * 10 + remainder;
        n /= 10;
    }
    print(rev);
}
```

Note: No need of counter increment for declarations and scanf() and count variable printf() statements.

Input:

A positive Integer n

Output:

Print the value of the counter variable

```
1 #include <stdio.h>
2
3 int main() {
4     int n, rev, remainder, counter = 0;
5
6     scanf("%d", &n);
7
8     rev = 0;
9     counter++;
10
11     while (n != 0) {
12         counter++;
13
14         remainder = n % 10;
15         counter++;
16
17         rev = rev * 10 + remainder;
18         counter++;
19
20         n /= 10;
21         counter++;
22     }
23
24     counter++;
25
26     counter++;
27
28     printf("%d", counter);
29
30     return 0;
31 }
```

	Input	Expected	Got	
✓	12	11	11	✓
✓	1234	19	19	✓

Passed all tests! ✓

Correct

Marks for this submission: 1.00/1.00.